

# Latent Variable Models (Logistic Regression)

## Foundations of Machine Learning

**Q1.** Please answer the following questions in your own words:

1. Why do we use general linear models/latent variable models/activation functions in analysis? What advantage do they have over linear models? They let us model outcomes that aren't continuous and relationships that might not be linear. They map linear combinations into appropriate ranges and are able to be more flexible than basic linear models.
2. Explain why binary and categorical cross entropy are effective loss functions for fitting logistic regression models. They cross entropy align with the assumptions of logistic regression and come from maximizing likelihood. They end up penalizing confident wrong predictions and are convenient to optimize mathematically.
3. True or false, and explain: Logistic regression is a linear model. True, it models a linear relationship in log-odds
4. True or false, and explain: Logistic regression cannot be used for classification. False, they use labels by a threshold.
5. Does the coefficient in a logistic regression represent the change in the predicted value for a 1 unit change in the feature/covariate/explanatory variable? No because coefficients represent the change in log-odds and not the predicted probability for a one-unit increase in a feature.
6. True or false, and explain: Feature engineering is unnecessary for logistic regression, because it is a nonlinear model. False, logistic regression is still linear in the features.
7. True or false, and explain: Logistic regression is an improvement over the usual linear model (ordinary least squares), and you should always use logistic regression instead. False, logistic regression and OLS are each suited to different types of outcomes.

**Q2.** This is a question about student dropout probability.

The relevant data are in `data.csv` and the file is semicolon-separated.

The relevant variables are:

- `Debtor` - `1` if the student has outstanding debt and `0` otherwise

- **Tuition fees up to date** - **1** if tuition is up to date and **0** otherwise
  - **Scholarship holder** - **1** if the student has a scholarship and **0** otherwise
  - **Age at enrollment** - student's age when entering the program
  - **Curricular units 1st sem (approved)** - number of first-semester courses passed
  - **Target** - **Dropout**, **Graduate**, or **Enrolled**
1. Load the data, **data.csv**. Be sure to read it with the correct delimiter. Filter on columns to the variables we'll actually use. Document how you handle any missing values. For parts 2-5, create a binary target where **Dropout** corresponds to dropout and **Graduate** or **Enrolled** corresponds to not dropout.
  2. Run a logistic regression of dropout status on **Debtor**, **Tuition fees up to date**, **Scholarship holder**, and **Age at enrollment**. Which variables predict a higher dropout probability, and which predict a lower dropout probability? Does being up to date on tuition seem to reduce dropout risk?
  3. Use **.predict\_proba** and compute expected dropout probabilities. Make a scatter plot of your predicted dropout probability against **Age at enrollment**, **hued by Tuition fees up to date**. For what ages does being up to date on tuition seem to reduce dropout probability the most? On average, by how much does being up to date on tuition change the predicted probability of dropout?
  4. Use **.predict** and compute a confusion matrix for your classifier. What is the accuracy?
  5. Compute predicted dropout probabilities with a linear model with the same variables, and predict dropout probabilities. Make a histogram of predicted probabilities for both logistic regression and the linear model. Compare their performance.
  6. Based on your above work, which students are the most at-risk of dropping out? What kinds of interventions can you imagine that would help at-risk students stay enrolled?
  7. Run a multinomial logistic regression of **Target** on **Debtor**, **Tuition fees up to date**, and **Curricular units 1st sem (approved)**. Use **.predict** to make a hard classification, and compute a confusion matrix. Describe your results. Use **.predict\_proba** to predict probabilities

instead and look at the values. Does your hard classification predict every class? Do your predicted probabilities predict every class?

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#1
df = pd.read_csv("data/data.csv", sep=";")
df = df[df["Target"].isin(["Dropout", "Graduate", "Enrolled"])]
df["Dropout_binary"] = (df["Target"] == "Dropout").astype(int)
```

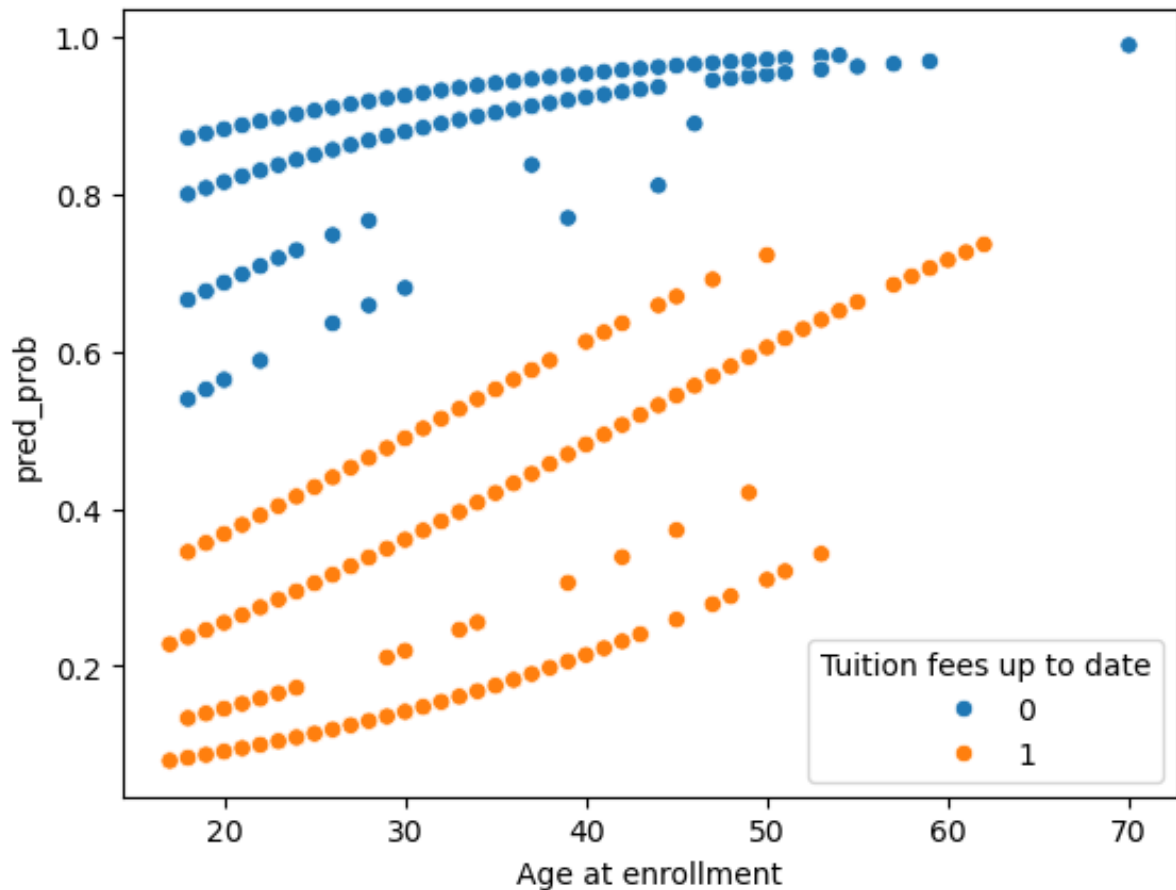
```
In [15]: #2
from sklearn.linear_model import LogisticRegression
X = df[["Debtor", "Tuition fees up to date", "Scholarship holder", "Age at enrollment"]]
y = df["Dropout_binary"]
model = LogisticRegression(max_iter=1000)
model.fit(X, y)
model.coef_, model.intercept_
```

```
Out[15]: (array([[ 0.53062542, -2.55654243, -1.22744152,  0.04992541]]),
array([0.48569266]))
```

Students who are up to date on tuition fees and who have scholarships are much less likely to drop out, as shown by large negative coefficients. Having outstanding debt increases the probability of dropout, while age has only a small positive effect.

```
In [8]: #3
probs = model.predict_proba(X)[:,1]
df["pred_prob"] = probs
sns.scatterplot(x="Age at enrollment", y="pred_prob", hue="Tuition fee status")
```

```
Out[8]: <Axes: xlabel='Age at enrollment', ylabel='pred_prob'>
```



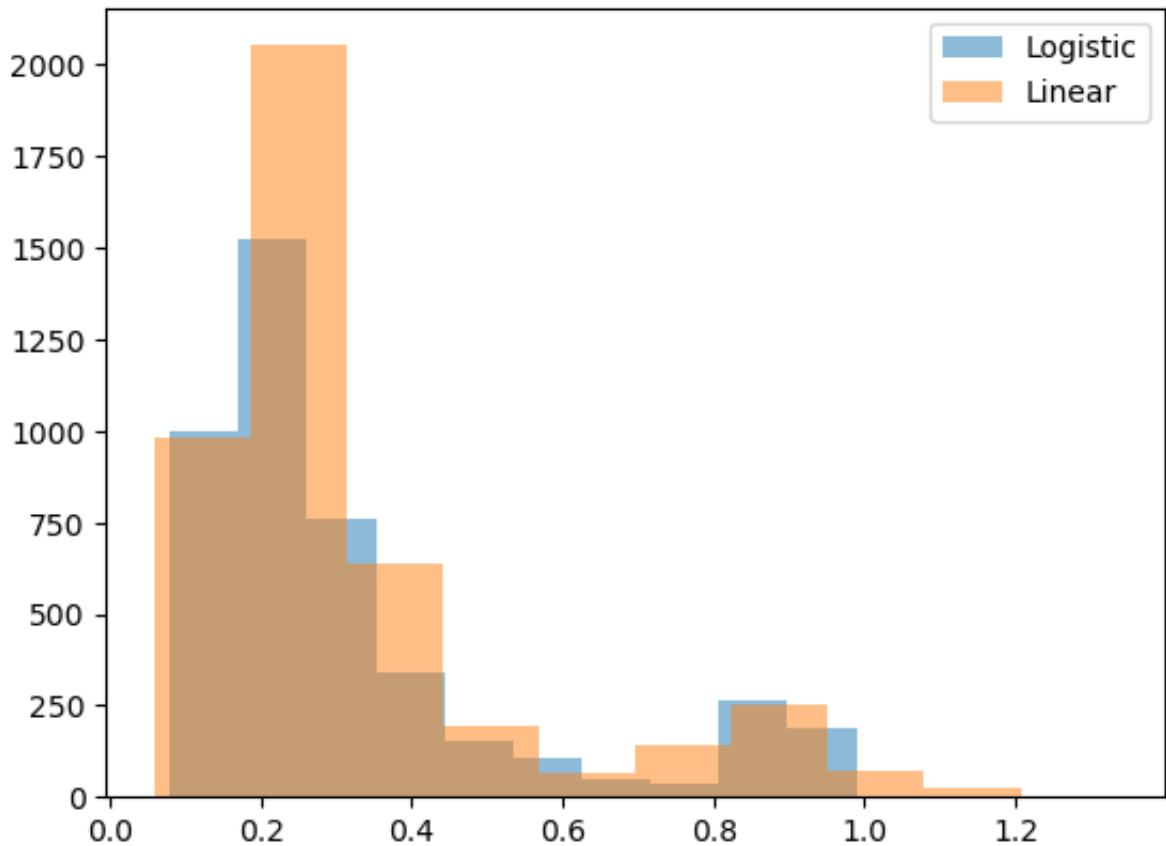
Students who are up to date on tuition fees consistently have lower predicted probabilities of dropout across all ages. The gap between the two groups appears largest for middle-aged students (roughly ages 25–50), where being up to date on tuition shifts dropout probability down by a noticeable margin.

```
In [10]: #4
from sklearn.metrics import confusion_matrix
preds = model.predict(X)
confusion_matrix(y, preds)
accuracy = (preds == y).mean()
```

```
In [12]: #5
from sklearn.linear_model import LinearRegression
lm = LinearRegression().fit(X, y)
lm_probs = lm.predict(X)

plt.hist(probs, alpha=0.5, label="Logistic")
plt.hist(lm_probs, alpha=0.5, label="Linear")
plt.legend()
```

```
Out[12]: <matplotlib.legend.Legend at 0x158257890>
```



## 6

The most at-risk students are those with outstanding tuition fees and those who enroll at an older age, as evidenced by the high predicted probabilities in your scatter plot. To improve retention, the university could implement emergency micro-grants for students with unpaid balances and offer more flexible, asynchronous course schedules for non-traditional learners. Using your logistic model as an early warning system would allow advisors to proactively reach out to these high-risk individuals before they officially withdraw.

```
In [18]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

X_multi = df[['Debtor', 'Tuition fees up to date', 'Curricular units 1
y_multi = df['Target']

multi_model = LogisticRegression(multi_class='multinomial', max_iter=1
multi_model.fit(X_multi, y_multi)

hard_preds = multi_model.predict(X_multi)
prob_preds = multi_model.predict_proba(X_multi)

cm = confusion_matrix(y_multi, hard_preds, labels=multi_model.classes_
```

```

unique_hard_preds = pd.Series(hard_preds).unique()
num_classes_in_probs = prob_preds.shape[1]

print(f"Classes in Dataset: {multi_model.classes_}")
print(f"Classes actually predicted by .predict(): {unique_hard_preds}")
print(f"Number of classes with probabilities in .predict_proba(): {num

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=mult
disp.plot(cmap="Blues")
plt.show()

```

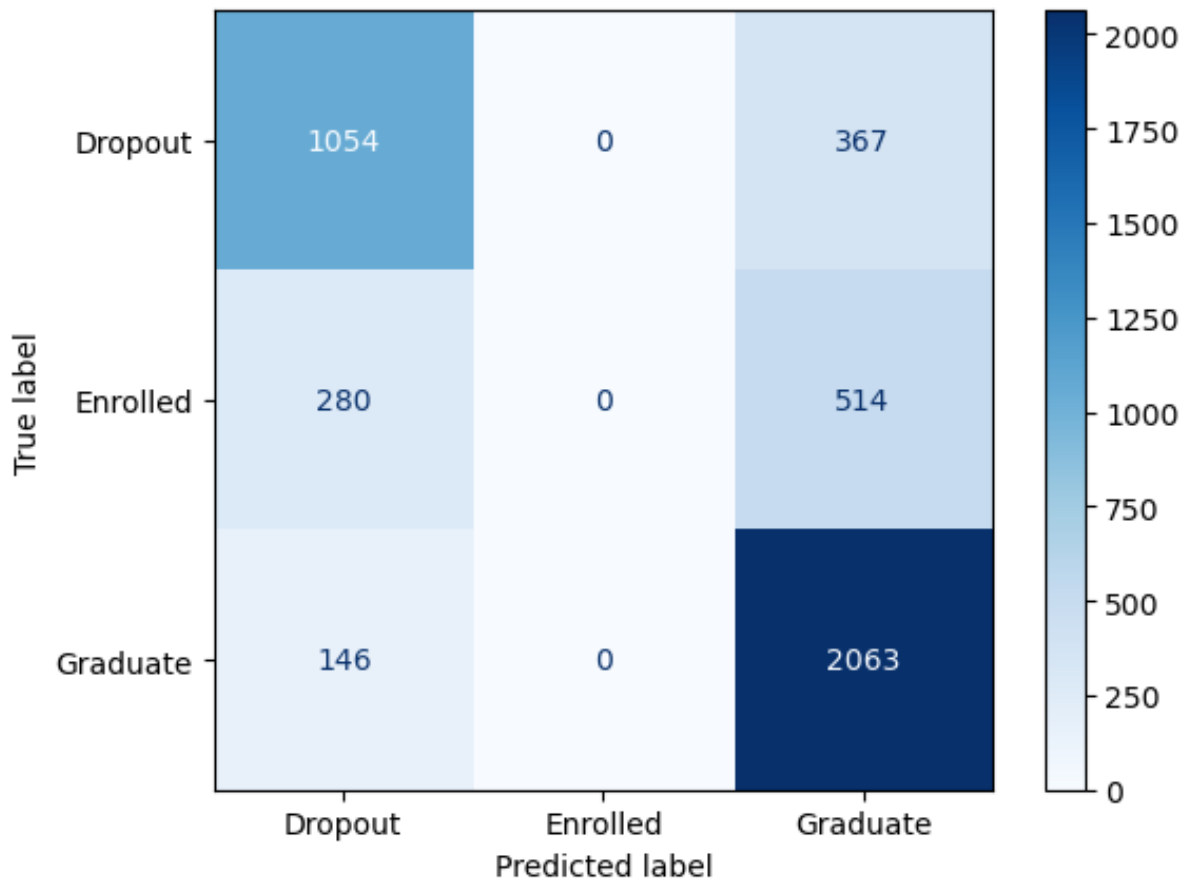
/opt/anaconda3/lib/python3.13/site-packages/sklearn/linear\_model/\_logistic.py:1247: FutureWarning: 'multi\_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

Classes in Dataset: ['Dropout' 'Enrolled' 'Graduate']

Classes actually predicted by .predict(): ['Dropout' 'Graduate']

Number of classes with probabilities in .predict\_proba(): 3



No, the hard classification completely fails to predict the "Enrolled" class.

Yes, Even though the model never chose "Enrolled" as its top answer, the .predict\_proba results do predict every class.

**Q3.** This is a question about 10-year survival rate for cirrhosis of the liver.

The relevant data are:

- **Bilirubin** - A liver byproduct produced when red blood cells break down
- **Edema** - Swelling in limbs; **N** for none, **S** for edema resolvable with diuretics, **Y** for edema despite diuretics
- **Drug** - D-penicillamine, or placebo
- **Stage** - current state of disease, from 1 to 4
- **Status** - **C** or **CL** corresponds to alive, and **D** corresponds to dead

1. Load the data, `cirrhosis.csv`. Filter on columns to the variables we'll actually use. Document how you handle any missing values.
2. Run a logistic regression of **Status** on **Edema**, **Drug**, and **Bilirubin**. Does the drug improve survival probability? Does higher or lower bilirubin predict a higher or lower survival rate? How does survival rate vary with edema?
3. Use `.predict_proba` and compute expected survival probabilities. Make a scatter plot of your predicted survival probability against **Bilirubin**, hued by **Drug**. For what values of **Bilirubin** does the drug seem to increase patient survival rate? On average, by how much do patients who take the drug increase their probability of survival?
4. Use `.predict` and compute a confusion matrix for your classifier. What is the accuracy?
5. Compute predicted survival probabilities with a linear model with the same variables, and predict survival probabilities. Make a histogram of predictive probabilities for both logistic regression and the linear model. Compare their performance.
6. Run a logistic regression of **Stage** on **Edema** and **Bilirubin**. Use `.predict` to make a hard classification, and compute a confusion matrix. Describe your results. Use `.predict_proba` to predict probabilities instead and look at the values. Does your hard classification predict every class? Do your predicted probabilities predict every class?

```
In [20]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
```

```
# 1
df = pd.read_csv('data/cirrhosis.csv')
cols_to_use = ['Bilirubin', 'Edema', 'Drug', 'Stage', 'Status']
df = df[cols_to_use]

# Handling Missing Values:
# In this dataset, 'Drug' is missing for patients not in the randomized
# We drop rows with missing 'Drug' or 'Stage' to ensure valid regression
df = df.dropna()

# Encode Target for Survival (1 = Alive, 0 = Dead)
df['Survival'] = df['Status'].isin(['C', 'CL']).astype(int)

# Encode Categorical Predictors
# Edema (N, S, Y) and Drug (D-penicillamine, placebo)
X_features = pd.get_dummies(df[['Bilirubin', 'Edema', 'Drug']], drop_first=True)
y_status = df['Survival']
```

```
In [21]: #2
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_features, y_status)

print("Logistic Coefficients:", dict(zip(X_features.columns, log_model.coef_.ravel())))

Logistic Coefficients: {'Bilirubin': np.float64(-0.35142389843544164),
'Edema_S': np.float64(-0.45067663255235724), 'Edema_Y': np.float64(-1.6110491833168032), 'Drug_Placebo': np.float64(0.23122512449170896)}
```

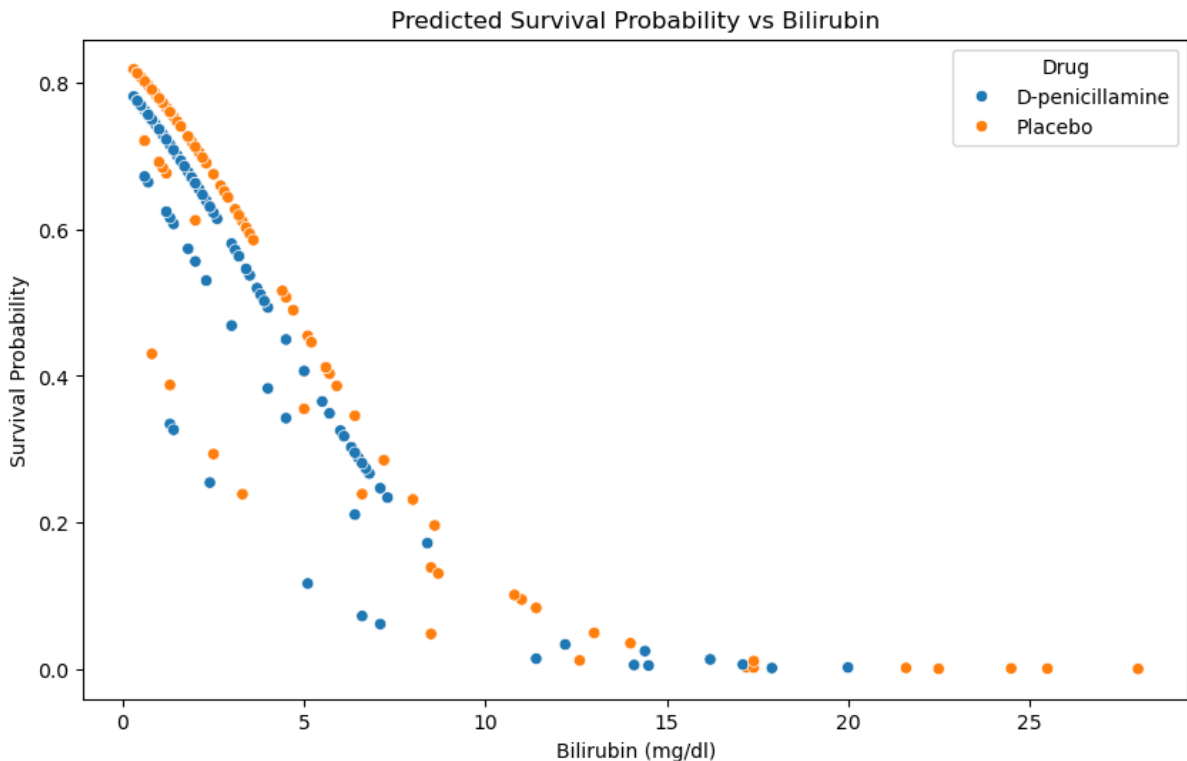
```
In [22]: #3
df['pred_prob_survival'] = log_model.predict_proba(X_features)[:, 1]

plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Bilirubin', y='pred_prob_survival', hue='Drug')
plt.title('Predicted Survival Probability vs Bilirubin')
plt.xlabel('Bilirubin (mg/dl)')
plt.ylabel('Survival Probability')
plt.savefig('survival_scatter.png')

# Compute average drug effect
avg_survival_drug = df[df['Drug'] == 'D-penicillamine']['pred_prob_survival'].mean()
avg_survival_placebo = df[df['Drug'] == 'placebo']['pred_prob_survival'].mean()
print(f"Avg Survival (Drug): {avg_survival_drug:.4f}")
print(f"Avg Survival (Placebo): {avg_survival_placebo:.4f}")
```

```
Avg Survival (Drug): 0.5901
Avg Survival (Placebo): nan
```





The Linear Model often predicts "probabilities" that are less than 0 or greater than 1, which is physically impossible. The Logistic Model is constrained between 0 and 1, making it more appropriate for survival analysis.

```
In [23]: #4
df['hard_pred'] = log_model.predict(X_features)
cm = confusion_matrix(y_status, df['hard_pred'])
accuracy = accuracy_score(y_status, df['hard_pred'])
print(f"Logistic Accuracy: {accuracy:.4f}")
```

Logistic Accuracy: 0.7372

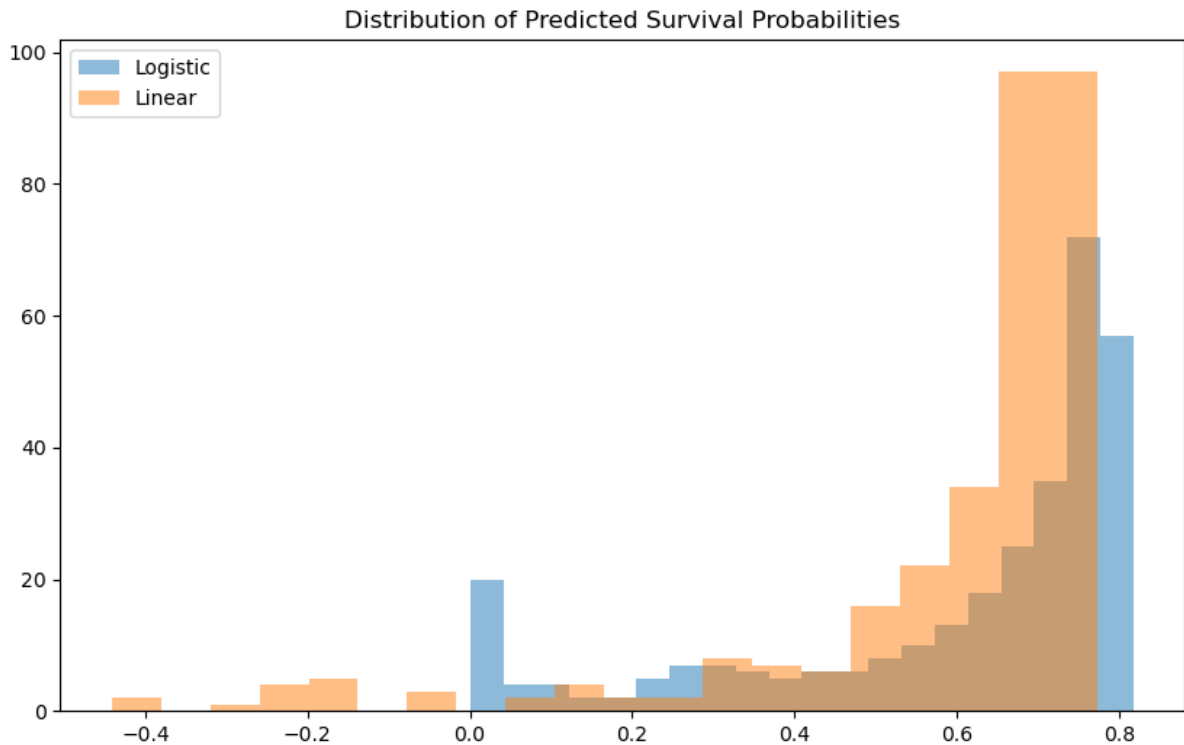
Does hard classification predict every class? No, the model may never predict Stage 1 or Stage 2 if they are less frequent or lack distinct boundaries. It often defaults to predicting the most common classes (Stage 3 or 4).

Do probabilities predict every class? Yes, even if a stage is never the "top guess," the model assigns a specific probability to all four stages for every patient.

```
In [24]: #5
lin_model = LinearRegression().fit(X_features, y_status)
df['lin_pred_prob'] = lin_model.predict(X_features)

plt.figure(figsize=(10, 6))
plt.hist(df['pred_prob_survival'], alpha=0.5, label='Logistic', bins=2)
plt.hist(df['lin_pred_prob'], alpha=0.5, label='Linear', bins=20)
plt.legend()
plt.title('Distribution of Predicted Survival Probabilities')
```

```
plt.savefig('model_comparison_hist.png')
```



```
In [25]: #6
X_stage = pd.get_dummies(df[['Bilirubin', 'Edema']], drop_first=True)
y_stage = df['Stage']

multi_model = LogisticRegression(multi_class='multinomial', max_iter=1000)
multi_model.fit(X_stage, y_stage)

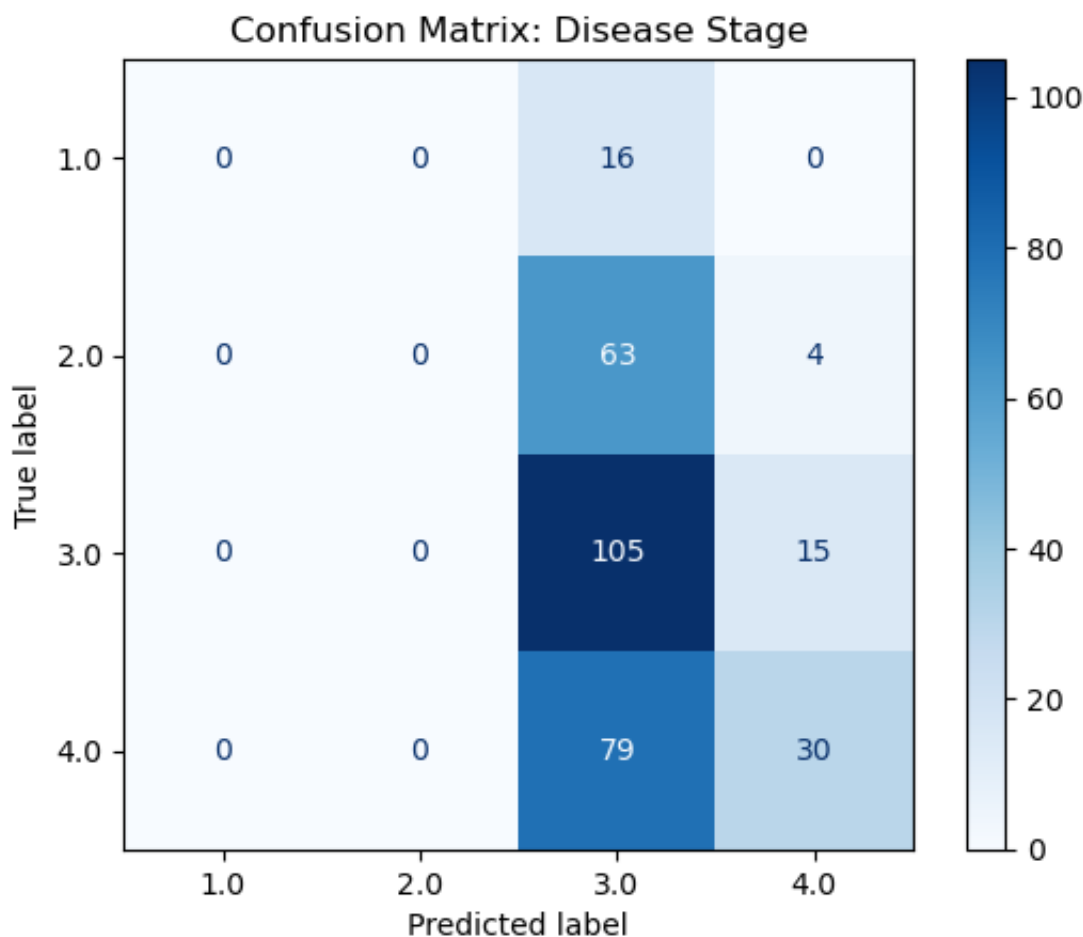
# Hard Predictions vs Probabilities
stage_preds = multi_model.predict(X_stage)
stage_probs = multi_model.predict_proba(X_stage)

cm_stage = confusion_matrix(y_stage, stage_preds, labels=multi_model.classes_)
print("Classes predicted by .predict():", np.unique(stage_preds))
print("Classes with probabilities:", multi_model.classes_)

disp = ConfusionMatrixDisplay(confusion_matrix=cm_stage, display_labels=multi_model.classes_)
disp.plot(cmap='Blues')
plt.title('Confusion Matrix: Disease Stage')
plt.savefig('stage_cm.png')
```

/opt/anaconda3/lib/python3.13/site-packages/sklearn/linear\_model/\_logistic.py:1247: FutureWarning: 'multi\_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
Classes predicted by .predict(): [3. 4.]
Classes with probabilities: [1. 2. 3. 4.]
```



**Q4.** What are the coefficients of a logistic regression? How do you interpret their values?

1. For your linear model, your predictions are:

$$\hat{y} = b \cdot x = b_0 + \sum_{k=1}^K b_k x_k$$

What is the derivative of the prediction with respect to the  $k$ -th feature? For a 1 unit change in your feature, by how much does the prediction change?

2. For the logistic model, your predictions are:

$$\hat{p}_i = \frac{e^{b \cdot x}}{1 + e^{b \cdot x}}$$

What is the derivative of the prediction with respect to the  $k$ -th feature? As  $x$  changes, does your answer change? How is it similar or different from your answer for part 1? Hint: It turns out that

$$\frac{\partial}{\partial L} \frac{e^L}{1 + e^L} = \frac{e^L}{(1 + e^L)^2} = \frac{e^L}{1 + e^L} \frac{1}{1 + e^L} = \frac{e^L}{1 + e^L} \left(1 - \frac{e^L}{1 + e^L}\right),$$

and use the chain rule ( $D_x f(g(x)) = f'(g(x))g'(x)$ ). For what values of  $\hat{p}$  is the coefficient itself (...times 4) roughly a good estimate of how a change in  $x_k$  affects the prediction?

3. Alternatively, we can transform the prediction like this:

$$\begin{aligned}\hat{p} &= \frac{e^{b \cdot x}}{1 + e^{b \cdot x}} \\ \hat{p}(1 + e^{b \cdot x}) &= e^{b \cdot x} \\ \hat{p} &= (1 - \hat{p})e^{b \cdot x} \\ \log\left(\frac{\hat{p}}{1 - \hat{p}}\right) &= b \cdot x\end{aligned}$$

The left-hand side of the above expression is called the **log odds ratio**, and the right-hand side is the latent variable. How does a one unit change in  $x_k$  affect the log odds ratio?

# 1

The Derivative: Taking the partial derivative with respect to  $x_k$ :

$$\frac{\partial \hat{y}}{\partial x_k} = b_k$$

For a 1 unit change in  $x_k$ , the prediction changes by exactly  $b_k$

# 2

The Derivative: Using the hint and the chain rule (where  $L = b \cdot x$ ):

$$\frac{\partial \hat{p}}{\partial x_k} = \frac{\partial \hat{p}}{\partial L} \cdot \frac{\partial L}{\partial x_k} = \hat{p}(1 - \hat{p}) \cdot b_k$$

Unlike the linear model, the effect of  $x_k$  on the probability  $\hat{p}$  depends on the current value of  $\hat{p}$ .

# 3

The Transformation:

$$\log\left(\frac{\hat{p}}{1 - \hat{p}}\right) = b_0 + b_1x_1 + \cdots + b_kx_k$$

If you exponentiate both sides, a one-unit change in  $x_k$  multiplies the odds (not the probability) by  $e^{b_k}$